



# >\_ FREMTIDENS APPLIKASJONER LAGES FOR MASKINER

Hvordan bygge og forstå fremtidens teknologi

[00 / AGENDA.md]

- Eksempler på maskindrevne applikasjoner
- Hvordan de kan bygges - metode og verktøy
- Refleksjoner

# >\_ FREMTIDENS APPLIKASJONER LAGES FOR MASKINER

## Eggformete egeninteresser

|                                  |           |
|----------------------------------|-----------|
| Lønn                             | 1 871 515 |
| Opsjonsytelser (naturalytel...   | 27 026    |
| Rentefordel rimelig lån i arb... | 1 280     |
| Forsikring betalt av arbeids...  | 7 836     |

 [Åpne og endre](#)

## Betalt telefon og bredbånd (elektronisk kommunikasjon)

|                     |       |
|---------------------|-------|
| Skattepliktig beløp | 4 392 |
|---------------------|-------|

 [Åpne og endre](#)

*Beløp vi bruker i skatteberegningen*

**Inntekt:** Lønn og 1 907 657  
naturalytelser med mer

**Fradrag:** Rentefordel rimelig 1 280  
lån i arbeidsforhold

**Inntekt:** Fordel elektronisk 4 392  
kommunikasjon

[Legg til lønn og tilsvarende ytelser](#)

[Stig.Lau@Skatteetaten.no](mailto:Stig.Lau@Skatteetaten.no)

## Fagforeningskontingent

### Fagforeningskontingent

|                               |       |
|-------------------------------|-------|
| Beløp (innbetalt)             | 5 208 |
| Fradrag for fagforeningsko... | 5 208 |

 [Åpne og endre](#)

*Beløp vi bruker i skatteberegningen*

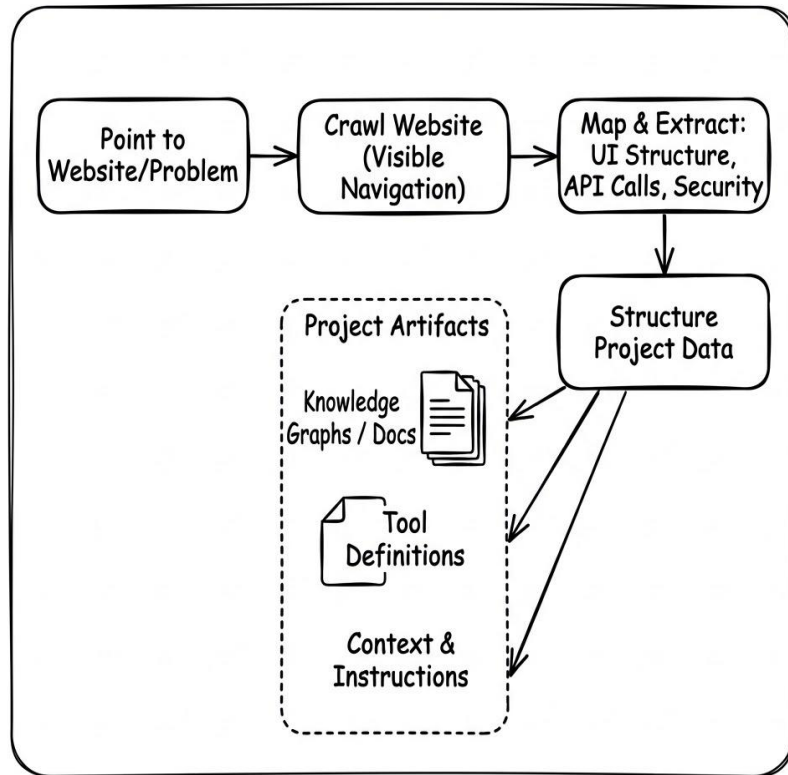
**Fradrag:** 5 208  
Fagforeningskontingent

## Ø1 // OLD SCHOOL HACKING

- Hvor er informasjonen og aksesspunktene du vil nå, hvordan få tilgang?
- Hva kan du måle fra eksisterende infrastruktur?
- Hvor mye betyr erfaring?

## Ø2 // SAMLE INFORMASJON / KCP TRIAGE

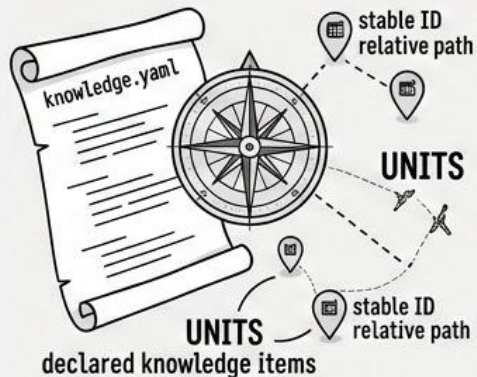
- Et system for å kartlegge spesifikke websider, gjennom synlig navigasjon, Playwright.
- Lagre observert struktur, oversikt og (REST) API-kall.
- Informasjonen skal lagres i en prosjektkatalog: KCP-format (Knowledge Concept Protocol format, og Skills, samt en katalogstruktur med Claude.md



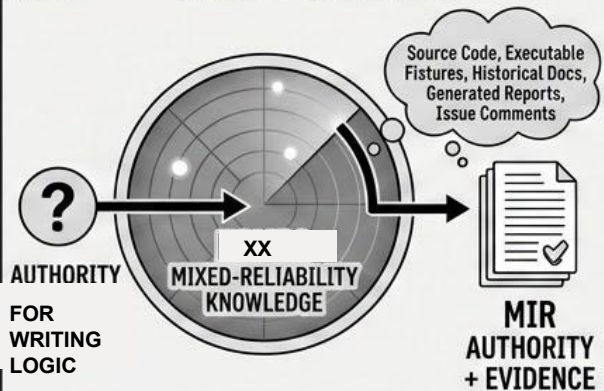


## ILLUSTRATED TRIAGE FLOW

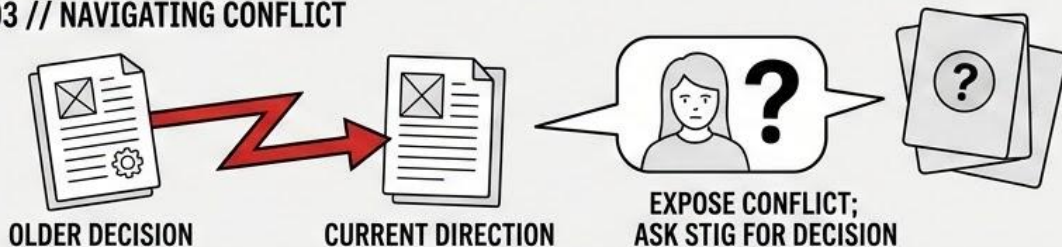
### 01 // THE KCP MANIFEST MAP



### 02 // | xx KNOWLEDGE ROUTING



### 03 // NAVIGATING CONFLICT



## NORMATIVE KCP ANATOMY

### NORMATIVE KCP UNIT ANATOMY

#### MINIMUM UNIT REQUIREMENTS

|          |                           |
|----------|---------------------------|
| id       | stable identifier         |
| path     | relative content location |
| intent   | question or task answered |
| scope    | applicability             |
| audience | relevant people/agents    |

#### UNIT DECLARATIONS & RELATIONSHIPS

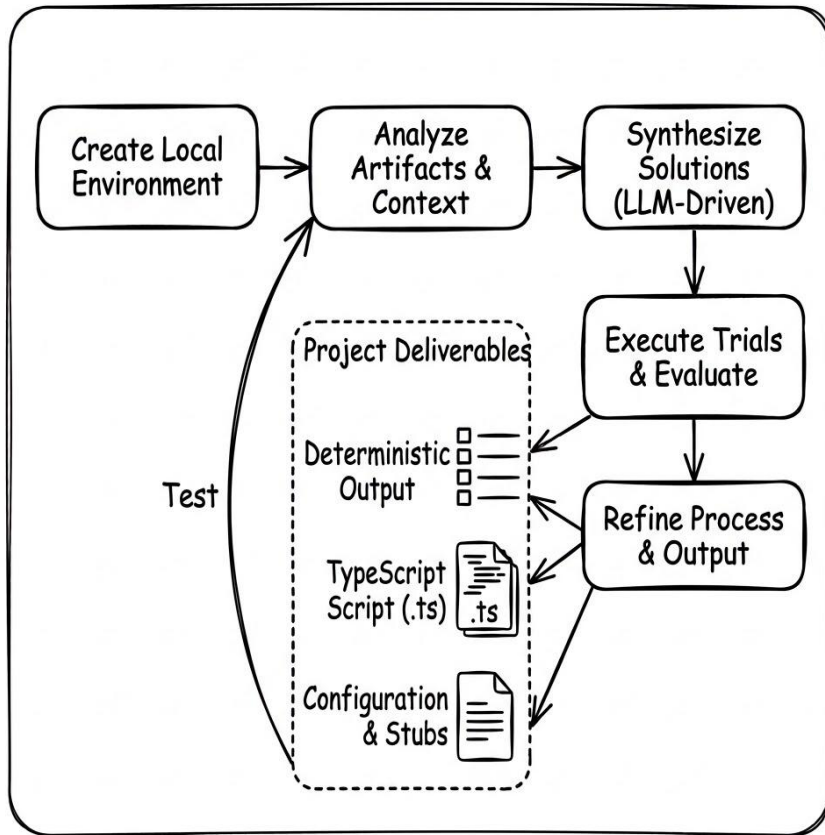
|              |                                                                            |             |
|--------------|----------------------------------------------------------------------------|-------------|
| ⚡ triggers   | stable identifier                                                          |             |
| 📦 kind       | knowledge, schema, policy, service, executable material, skills, playbooks |             |
| 💡 hints      | token estimates                                                            |             |
| 👤 depends_on | supersedes                                                                 | contradicts |
| 📦 context    | enables                                                                    |             |
| 👤 governs    | governs                                                                    |             |

## 03 // BYGGE KLIENTEN

- Start ny agent fra prosjektkatalog.
- Beskriv hva du ønsker av funksjonalitet.
- LLM leser skills og KCP dokumentasjon, men vet også hvordan den kan gjøre flere undersøkelser.
- Gjør forsøk og leverer rapporter/data.
- Dere evaluerer måloppnåelse/tester.
- Evaluer krystallisering av prosess i (.ts) script.
- Gjenta fra Beskrive ønsker.
- Make/Mise som CLI UI mot funksjoner

## 04 // NESTE STEG?

Web- og Mobil-Apper, SOAP og andre API'er,  
Database, Kompilert kode, Stormaskin, IOT dingser,  
LANtrafikk





## 01 // Prosess & Iterasjon

- **Beskriv prosessen:** Beskriv den eksisterende manuelle flyten, helst i så god detalj som mulig for agenten. Både helheten, men også en enkelt prosesser
- **Integrasjon:** For denne perioden, la agenten få tilgang til relevante filer og systemer så den kan utføre prosessen.
- **Manuell QA:** Kvalitetssikre resultatet; identifiser logiske feil, og diskuter problemstillinger kontinuerlig.

## 02 // Strukturert Læring

- **Ekstraher "Skills":** Be agenten om en strukturert oppsummering av samtalen, implementert som en spesifikk ferdighet.
- **Domenespesifikt fokus:** Avdekk hva som var unikt for akkurat denne prosessen som agenten ikke hadde kunnskap om fra før.
- **Kontekst-bevaring:** Ta oppsummeringen videre som initial kontekst i en helt ny økt for å unngå informasjonstap.

## 03 // Beskriv Helheten og Automatisk Tester

- **QC:** Brukes som valideringsgrunnlag for å verifisere retning

## 04 // Iterer

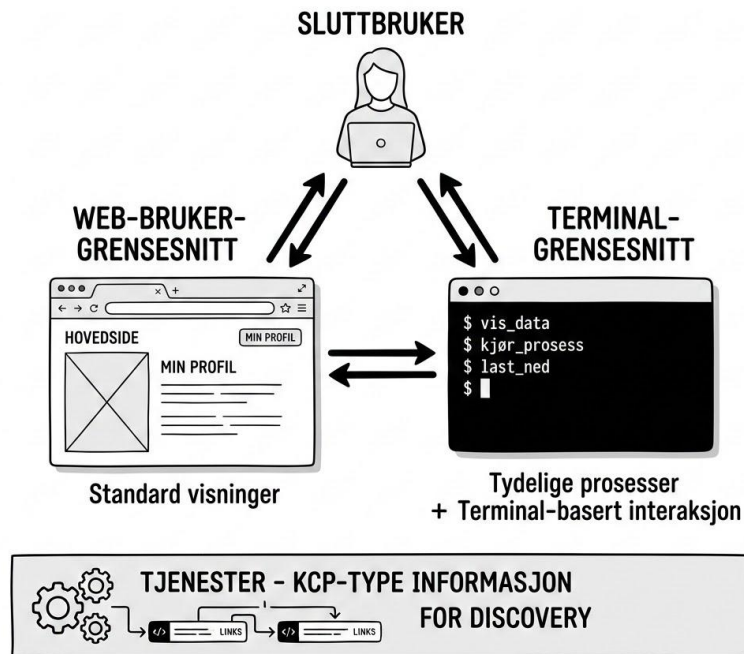
- Med skills i kontekst, prøv en annen problemstilling gjennom og verifiser verdi

## 05 // Krystalliser

- **Med utgangspunkt i skills:** Be agenten om en 50-80% skriptet, pragmatisk, fail-fast løsning for de vanligste brukstilfellene
- **Domenespesifikt fokus:** Avdekk hva som var unikt for akkurat denne prosessen som agenten ikke hadde kunnskap om fra før.
- **Kontekst-bevaring:** Ta oppsummeringen videre som initial kontekst i en helt ny økt for å unngå informasjonstap.

- Hybrid av standard visninger i UI og tydelige prosesser som kan gjennomføres
- Et chat interface for å få tilgang til de underliggende funksjonene, a-la dagens tilrettelagte chat-botter
- Mange vil tillate tilsvarende funksjonaliteter for outsourcing av agent til bruker
- La sluttbruker kjøre funksjoner, laste ned og manipulere data og la dem kombinere med data fra andre leverandører og lage nye visninger og prosesser
- Tjenester tilbyr KCP-type informasjon for discovery og forstå tjenestene
- Færre knapper - kontekst-basert informasjon - hjelpe bruker til å finne ut av muligheter

## FREMTIDENS UI - EN HYBRID VISJON





## [01 / ARKITEKTUR OG PARSING]

LLM og Agenter er et verktøy. LLM kommunikasjons protokollen, MCP, Harness, verktøykall er relativt enkle konsepter å få et begrep om.

## [02 / Sentrale konsepter]

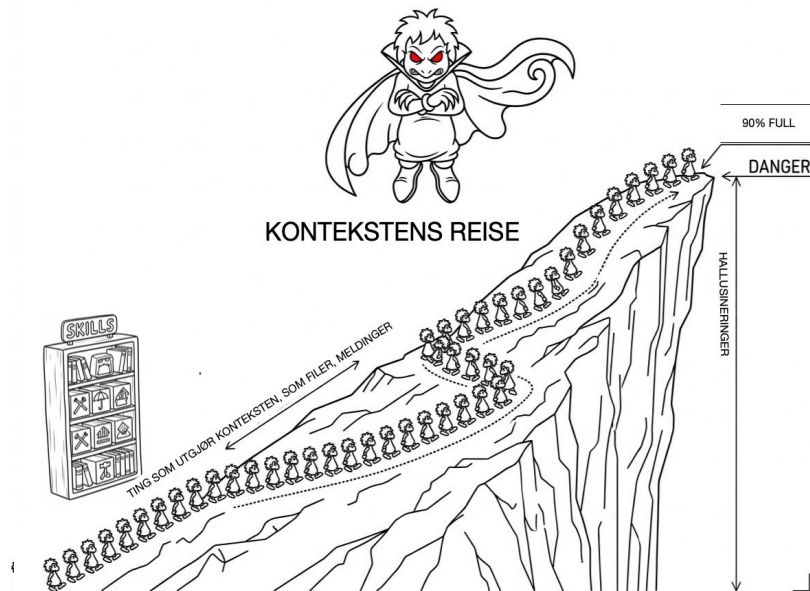
Samtaleloggen er det sentrale - konteksten. Alt i samtalen, dine meldinger (prompts), det LLMen har tenkt, innhold fra filer den har fått tilgang til, baseprompts etc sendes/evalueres for hver gang. Jo større konteksten er, jo mer inputtokens koster det. Produksjon av tekst koster mer (outputtokens). Skills er en teknikk for å dra ut informasjon fra basekonteksten ([claude.md](#)), som en bok i en bokhylle med forordet skrevet utenpå

## [03 / METADISKUSJONER]

LLM'en har ofte god kunnskap om egen teknologi og selvinnsikt i problemer relatert til seg selv. Prosessforbedring - diskuter problemer dere sammen har erfart, og spør hvordan forbedre. Finn den gode samtalen!

## [04 / LOKAL KJØRING, MODELLVALG og MCP]

Med llama.cpp, LM Studio (LLM App store) kan du kjøre relativt smarte LLMer lokal hardware med grei tokenfart. Bruk dyre modeller som Opus-nivå til å tenke strategi, omfattende endring, og levere rapporter som Sonnet-nivå kan implementere, gjerne som skills og kode. Billigere modeller til hverdags. MCP (Model Context Protocol) er fint for å eksperimentere med. Tankesettet fremmer drømmer men protokollen burde ikke være med til prod.







## 01 // KONTEKST SOM EN CHATLOGG

**Alt sendes på nytt:** Prompts, reasoning steps, verktøykall og resultater sendes som en samlet pakke hver runde.

**Eskalierende kostnad:** Jo større konteksten blir, desto dyrere blir hver input-token.

**En fast ressurs:** Kontekstvinduet fylles raskt opp. Noe må gjøres før det renner over.

## 02 // BEVARE DET VERDIFULLE

> **Compaction (Innebygd):** Komprimerer løpende. Billig, men **lossy**. Viktige detaljer og "arv" kan plutselig forsvinne uten varsel.

> **Forking (Ny gren):** Starter på nytt med et destillert sammendrag. Rent og kontrollert, men krever aktive arkitekturvalg.

## 03 // ARKITEKTURENS KONSEKVENSER

**Ingen auto-persistering:** Ingenting overlever sesjonen automatisk. Minne må eksplisitt lagres i et periodisk register (KCP-memory).

**Sammendrag-risiko:** En oppsummering laget for en spesifikk situasjon er sjelden dekkende for unntakene i neste runde.

## KRITISK RETNINGSLINJE & SIKKERHETSRISIKO

**Minnekorrumperting:** Det er en reell og overhengende fare for korrumperting av det delte minnet når flere uavhengige agenter skriver til og overskriver samme kontekstlager samtidig.

**Ved delt minne:** Bruk kontrollert skrivetilgang (kø eller låsing) og en klar strategi for feilhåndtering, eller risiker tapt eller korrupt prosessdata

# JZ<sup>26</sup> Periodisk minne, deling mellom agenter og håndtering av kontekstvinduer



Et av de største arkitektoniske problemene i dagens LLM-drevne systemer er at kontekstvinduet er begrenset og fylles opp raskt. Skal konteksten vare lenger enn én enkelt sesjon, må man velge en strategi for forking og compaction.

Dette minnet er designet for å overleve hard komprimering (compaction) uten tap av semantisk mening, og kan fritt og asynkront deles på tvers av ulike agenter og LLM-instanser.

Med dette delte minnet på plass, kan flere agenter samarbeide aktivt og intelligent gjennom delegering av oppgaver, virtuelle round-table-diskusjoner, asynkrone fora, og ved direkte sending av optimaliserte prompts og meldinger til hverandre.

**ALTERNATIVER:** KCP-memory, Claude Memory, Obsidian

## SKILLS

STATISKE FERDIGHETER

## PERIODISK MINNE

REALTIDSDELING

## SYSTEMLÆRING

SYNTHESIS/ARKIV

## harness\_responsibilities.log

Seletøyet håndterer kommunikasjonen med LLMen. Toolsene og dets integrasjoner er dets armer og muliggjør integrasjoner mot applikasjonen.

All data går fra backend via Seletøyet til LLM, og alle svar returneres gjennom samme rør.

### -- KRITISKE SYSTEMOPPGAVER --

- > Parse innkommende kommandoer
- > Filtrere tilganger (sluttbrukers tillatelser)
- > Styre funksjonskall (tools) med tilgangskontroll
- > Overvåke delt minne & minne-compaction

## extensions\_and\_tools.md

*"Eget seletøy kan skrive .js-funksjoner direkte i nettleseren for utvidelse."*

---

**Claude Code** er i essens et glorifisert Seletøy med CLI-tilgang.

Ting å tenke over:  
Meldingshistorikk, compaction strategi, base-prompt,

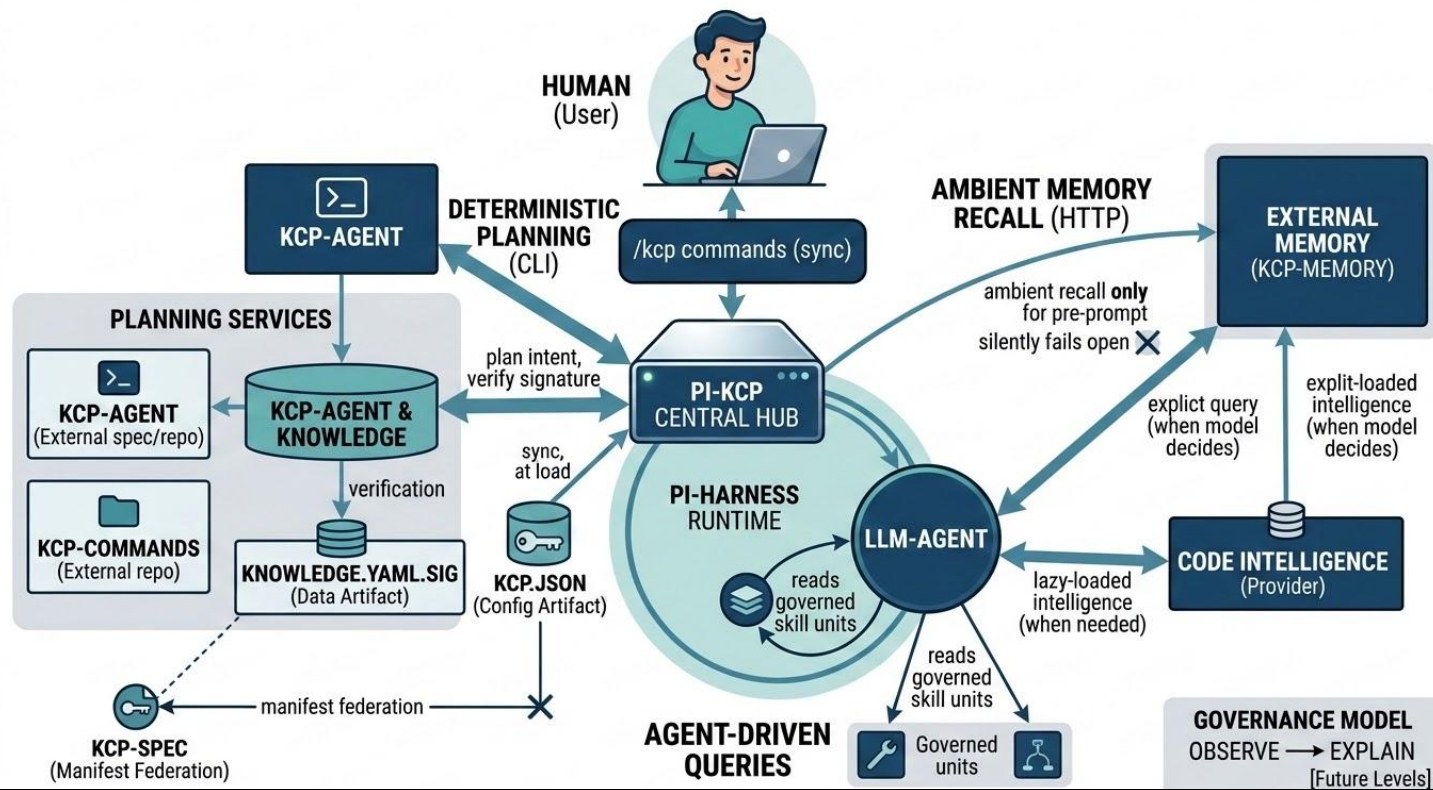
Vurder: Skills kunnskaps-bibliotek, minne-

### -- SIKKERHET --

Strup ned aksessflater og blast-radie.  
Vurder som DMZ (i docker)

# JZ<sup>26</sup> PI-KCP gir agenten din hukommelse og grenser

Agenten husker hva prosjektet har bestemt før, og den vet hva den faktisk har lov til å gjøre





## Ø1 // PROBLEM & LØSNING

- Vår utviklerprosess er tett knyttet opp mot Jira, git og kubernetes.
- En issue omfatter som regel flere git-prosjekter samtidig
- I en folder med vår-prosess skill, be om å laste ned jiraens git brancher.
- Deployment-kommando sjekker om nåværende versjon av hvert prosjekt er bygget på CI, deployert i k8s, og hvis ikke hvor det har gått galt på veien
- Samme struktur kan brukes for å gjøre PR evalueringer, få støtte i å se etter topp 10 problemer med endringer, teste noen andres kode i din k8s

## Ø2 // PROSESS

- En harness og enkapsulering som beskytter maskinen din og begrenser agenten; navikt/cplt
- Dytte KCP-triage foran for utforsking,
- KCP-SAFE for håndtering av integrasjoner og hemmeligheter.
- Skills for 1. Hånds lagring om kunnskap
- KCP for utvidet informasjonsstruktur





## SENTRAL PLASSERING (LEVERANDØREN BESTEMMER)

## [ DEN ARKITEKTONISKE DEBATTEN ]

Produktleverandører vil alltid argumentere for å koble LLM'en i midten av alt – integrert direkte på den sentrale meldingsbussen. Målet er uhemmet tilgang til alt av data og sanntidspåvirkning på alle systemvalg.

## [ RISIKO &amp; BIVIRKNINGER ]

- > **Passive operatører:** Dine ansatte blir passive ("dumme") operatører. Personlig erfaring, selvstendig tenkning og etablerte kvalitetssikringsprosesser tæres sakte ned.
- > **Passive operatører:** Dine ansatte blir passive ("dumme") operatører. Personlig erfaring, selvstendig tenkning og etablerte kvalitetssikringsprosesser tæres sakte ned.
- > **Token-sløsing:** Dyrebare tokens brennes kontinuerlig til evig tid for hver minste transaksjon på bussen.

## KONTROLL OVER HARNESS, TJENESTER &amp; VALG

## [ MOTVEKT ]

Tenk selv, eksperimenter lokalt og ta tilbake kontrollen. Dytt i stedet Agenten foran deg i prosessen, på kanten av systemet, i stedet for å gjøre deg avhengig av låste arkitekturer.

## [ ALTERNATIV STRATEGI ]

Du KAN begynne med en sentralt plassert under utvikling og krystalliserefunksjonalitet over tid

Et alternativ er at konsumentten eier LLM'en, du tjenesten og det Agenten trenger for å komme igang

## [ AUGMENTERT AGENTISK UTVIKLING ]

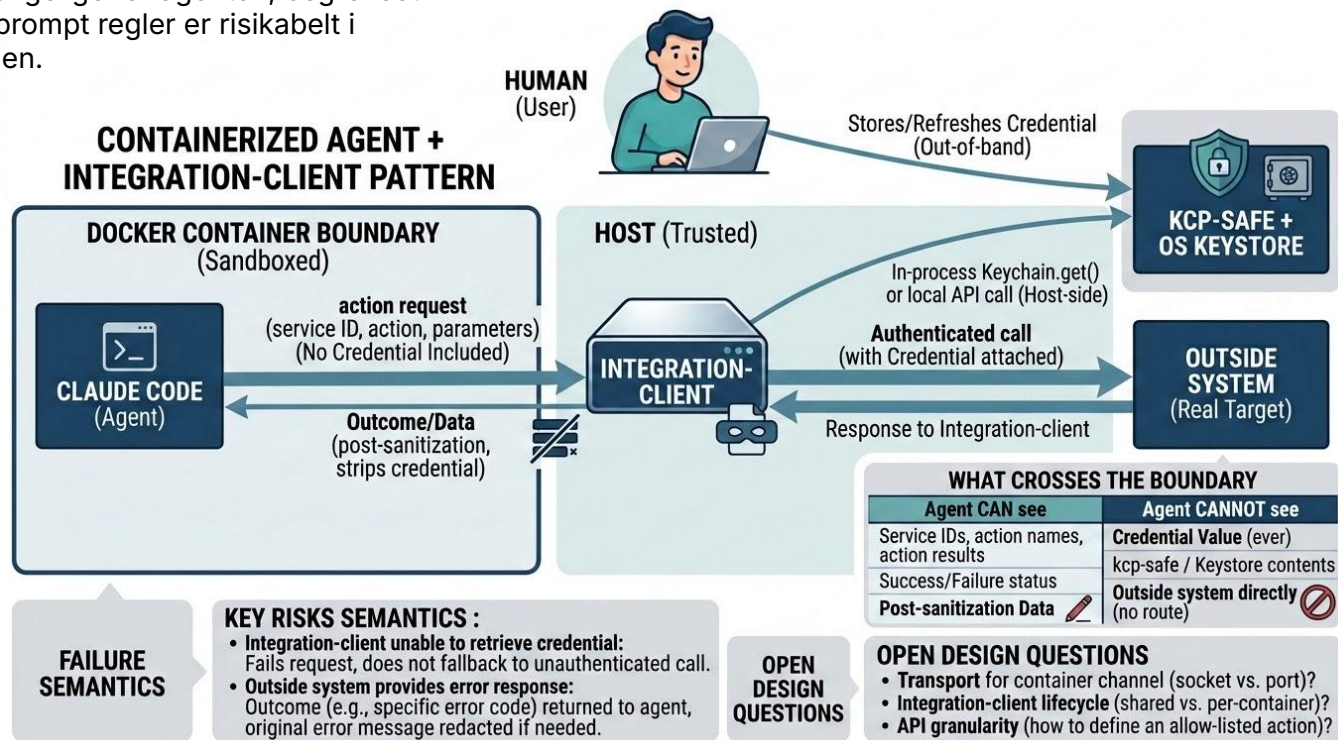
- > **Presisjon koder:** Hvis du klarer å beskrive et system presist, kan en agent med riktig oppsett kode det like bra som en dyr enterprise-leverandør.
- > **Pragmatisme:** Sikt på en 50-80% "fail-fast"-løsning i starten. Bygg ferdigheter først, og krystalliser script etter iterasjoner og reell testing.

# Beskytte hemmeligheter og tilganger fra agenten

Å gjøre hemmeligheter, integrasjoner og tilganger utilgjengelige for agenten, begrenset med prompt regler er risikabelt i lengden.

Behovet for hemmelighetene og tilgangene kan pakkes inn i scripts eller tjenester, der agenten har funksjonalitet som normal bruk og helse.

Disse integrasjonstjenestene kan igjen pakkes inn og flyttes utenfor containeren



## Ø1 // PROBLEMET

- Hver enkelt prosjektgruppe har en begrenset kjennskap til et sett av alle systemer i porteføljen. Ofte bundet til teknologi eller org
- Superbrukere og domeneeksperter kan ofte se sammenhenger, muligheter og begrensninger i domenet, men kan være begrenset til systemene de kjenner godt eller har ikke utvikler-kompetanse til å utforske systemer.

## Ø2 // FORARBEID

- Hvert enkelt system/prosjekts deltagere har beskrevet struktur, skills, skjema, relaterte beskrivende dokumenter for sine systemer.
- Denne informasjonen er tilgjengeliggjort gjennom et felles register med en teknologi a-la KCP for å gjøre det lett for agenter å konsumere den.

## Ø3 // DRØMMEN

- Domeneeksperten drømmer opp en tese om hva som skal til for å finne ...sannsynlige avvik som faller utenfor nåværende prosesser... og har relevante referanser som fødselsnummer etc å starte fra
- Agenten tolker drømmen, omsetter den i systemer som skal kalles, data hentes, manipuleres og verifiseres
- Script krystalliseres, løftes til høyere testsystem og testes UTEN LLM. Feil vaskes og returneres.
- Verifisere verdi før tung investering.
- Målet er en standardisert prosess som er kvalitetssikret av vanlige prosesser - utviklere, testere og juridisk.





# JZ<sup>26</sup> POTENSIELLE SIKKERHETSTRUSLER OG SÅRBARHETER SOM EN KONSEKVENNS AV AGENT-DREVET ARKITEKTUR FOR BÅDE KLIENT OG TJENER

## KLIENT- OG TJENERSIDETRUSLER

- Vi ER allerede under angrep fra autonome agenter.
- Det er en risiko hvis vi ikke utfører kontinuerlige agent-drevne penetrasjonstester på oss selv med alle modeller for å verifisere egen sårbarhet
- Et ondsindet LLM kan utføre grundig, utrettelig triage over tid for å kartlegge svakheter i systemet.
- En LLM i en tjeneste, vil være det naturlige punktet å angripe målrettet, overtale den å skifte side, bruke dens verktøy/tilganger og bryte ut
- De kan rette målrettede angrep eller skape støyende feil som er ekstremt vanskelig å oppdage.

## DATA- OG MINNEMANIPULASJON

- LLMer har en veldig dårlig evne til å skille mellom data i dens kontekst over tid. Compaction gjør det værre.
- Lagring og persistering av ufiltrert informasjon i agentens minne kan endre og manipulere hele tjenestens logikk.
- LLM'er elsker å løse utfordringer og finne mønstre i kaos – og utnytter gjerne dette.
- De har kapasitet til å desifrere råttall, binærdata og bytes for å omgå etablerte sikkerhetsbarrierer.
- Vi må beskytte LLM'en mot data som kommer utenfra, den henter selv på internett, men også dårlig data
- Hvis inputdata persisteres finnes det en sannsynlighet for data escape og at funksjonaliteten til agenten kan endres

## ! KRITISK RISIKO: PROMPT INJECTIONS

Tidligere lagret data kan inneholde skjulte **"LLM escape-kommandoer"** som ligger i dvale til de hentes opp og tolkes farlig andre steder i systemet – en trussel tilsvarende **log4j**-sårbarhetene.

**[ OPPSUMMERING ]****Ø1 // TA KONTROLL OVER SELETØYET**

Få total kontroll internt i egen organisasjon. Vær arkitektonisk villig til å koble deg helt løs fra spesifikke skyleverandører når det trengs.

**Ø2 // UNNGÅ MODELL-LÅSING**

Ikke forelsk deg i en spesifikk modell – leverandørinnlåsing (lock-in) kommer til å koste skjorta på sikt når prismodellene endres.

**Ø3 // SPØR HVORFOR AVGJØRELSE BLE TATT**

Forstå teknologien. Et kritisk spørsmål som alltid må besvares: Hvorfor tok agenten akkurat denne avgjørelsen?

Bevar den kritiske ingeniørsansen!

**VIKTIGSTE REGEL: ALDRI OUTSOURCE SYSTEMTENKING OG ARKITEKTURVALG TIL AUTONOME AGENTER!**

**[ SKATTEETATEN - VI REKRUTTERER! ]****Ø1 // TA KONTAKT PÅ VÅR STAND!**





## Ø1 // LABYRINTER & HONEYPOTS

> **Skjulte labyrinter:** Etabler kontinuerlige labyrinter i applikasjonsflyten. Mål responstider og navigasjonsmønstre for å avdekke uregulert, robotisk aktivitet.

> **Aktive Honeypots:** Fang fiendtlige agenter interne tilstander og tankeprosesser via KCP-protokollen og minneovervåking for dypere analyse.

## Ø2 // STRATEGISK KOSTNADSSTYRING

Trafikk rutes intelligent basert på oppgavens kompleksitet for å opprettholde et kostnadseffektivt forsvar:

**[High-Tier]** Avanserte LLM-er brukes til dyp sikkerhetsanalyse, refactoring og komplekse strategiske oppgaver.

**[Low-Tier]** Raske, rimelige modeller håndterer enklere, repetitive oppgaver og rutinemessig triage.

## Ø3 // BEGRENSNING OG ENKAPSULERING

- Ta kontroll på seletøyet
- Minimere verktøytilgang og tilganger
- CONTAINIFISERING - [github.com/navikt/cpl](https://github.com/navikt/cpl)
- Nettverksstyring / DMZ

KRITISK ARKITEKTURKRAV // STRENG KILDEKRITIKK

Signer all innkommende data kryptografisk med **Kilde**, **Tidsstempel** og **Verifikator** i tråd med KCP-standard.

# Hvorfor stokastiske agenter aldri må ta saksbehandleravgjørelser alene



Personsensitive data (PII) bør ikke sendes til eksterne LLMer over nettverk vi ikke har kontroll over og prosesseringsplattformer vi ikke 100% kontrollerer!

Stokastiske verktøy har en iboende svakhet og bør aldri ta selvstendige, juridiske valg.

## [Ø1] LOVMESSIG DETERMINISME & KVALITET

Ved klager på forvaltningsvedtak krever loven **full sporbarhet** og deterministisk kontroll på sentrale algoritmer. Vi kan aldri forsvare "Lotto-trekninger" i en lukket, svart boks.

Derfor må alle automatisk genererte kodesnutter og rutiner testes og godkjennes av menneskelige utviklere, testere og jurister etter eksisterende, strenge rutiner.

## [Ø2] FULLSTENDIG LOGGING & REPRODUSERBARHET

For å ivareta full kontroll over systemets oppførsel må alle forespørsler og svar loggføres med kritiske parametere:

- > Forespørsler, svar og nøyaktige tidspunkter
- > Spesifikke agent-IDer brukt under utføring
- > Nøyaktige systemkonfigurasjoner ved kjøring

# FREMTIDENS GRENSESNIITT FJERNER KNAPPER OG FOKUSERER PÅ KONTEKSTUELLE DATA ← Fikses!

## PROSESSEKVENNS // AUTONOM SAKSBEHANDLING

### 01 // MÅLDATABASER (KCP)

Seletøyet rettes direkte mot komplekse kilder: kjøretøyregisteret, grensepasseringer, banktransaksjoner, VPS og aksjeregisteret.

### 02 // SYNTETISKE TEST-STUBS

Lemenet finner API-er via KCP. Setter opp en autonom integrasjon i et trygt, syntetisk testmiljø med eksempeldata og stubs.

### 03 // DETERMINISTISK DRIFT UTEN LEMEN

Saksbehandler verifiserer 3-4 caser. Implementerer 50-80% skriptet løsning i test. Ingen Lemen kjører i endelig produksjon.

## KCP-INTEGRATION-HARNESS v1.0.4

```
$ lemen-harness --target kcp-registry
[INFO] API scan: Kjøretøy, VPS, Bank, Aksjer...
[INFO] Fant API-er i KCP. Etablerer test-stubs.
[OK] Syntetisk testmiljø operativt. Stubs klare.

$ lemen-harness --run-validation --cases 4
[WARN] Tvetydighet identifisert. Ber om saksbehandler.
[INPUT] Operatør ID lagt inn: "Exit-metafor aktiv"
[OK] 4 caser verifisert. Genererer dynamisk rapport.

$ lemen-harness --export-deterministic --output script.sh
[INFO] Eksporterer deterministisk flyt uten Lemen i prod.
[SUCCESS] script.sh (50-80% god nok) klargjort for test.
```

# AUTOMATISK KARTLEGGING OG TRIAGE MED PLAYWRIGHT OG KCP

## AUTOMATISK TRIAGE-MEKANIKK

### [01] MÅLRETTEDE FERDIGHETER (SKILLS)

Lemenet instrueres om sidens formål og nødvendige datapunkter som skal trekkes ut.

### [02] PLAYWRIGHT BRUKER-UTFORSKNING

Interagerer som en sluttbruker ved å låne aktiv sesjons-autentisering for å kartlegge REST-APIer.

### [03] CLAUDE CODE-GENERERING

Data lagres i KCP-format. Triage-resultatet er et ferdig konfigurert Claude Code-prosjekt klart til klientbygging.

root@lemen-harness:~

```
$ lemen-triage --target web.infra --skills ./kcp
[INFO] Initializing Playwright web driver...
[INFO] Active session authentication hijacked.
[INFO] Mapping endpoints & patterns:
• GET /api/v1/endpoints [REST-API]
• POST /auth/session-sync [SECURE]
[INFO] Structuring data into KCP schema...
[OK] KCP Schema saved: ./kcp/schema.json
[OK] Configured Claude Code client project.
$
```

## Ø1 // TANKER

- Det jeg leser ut fra analysen av Hugging-face angreps-analysen, er at de smarteste LLM'ene vi fortsatt ikke får tilgang til, er at de LLM'en fortsatt lever i samme paradigme som oss, bare litt mer ekstremt
- Gitt nok minne og compute (token/jobb), er de bedre til å styre egen retning, på vegne av subagenter
- Problemet er fortsatt tilgjengelig minne, compute og retningsstyring
- Hvis de kan skalere opp kan de i mye større grad rette 1000vis av agenter til å teste permutasjoner og finne sannsynlige angrepsvektorer
- De jobber fortsatt innenfor et paradigme vi kan forholde oss til som ingeniører:
  - Skrive en tese om problem
  - Delegere til subagenter og samle erfaringer
  - Gjøre 1000vis av forsøk ut ifra permutasjoner
  - Dokumentere erfaringer, for arbeid av senere agenter
  - Bygger opp lette varianter av jira, slack, git

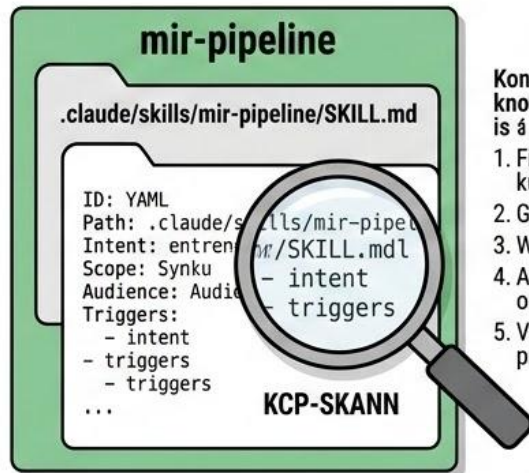




## KCP // OPERASJONELL MAP-STRUKTUR & PROSESSPLITT

### KCP DEFINISJON & KUNNSKAPSMAPING

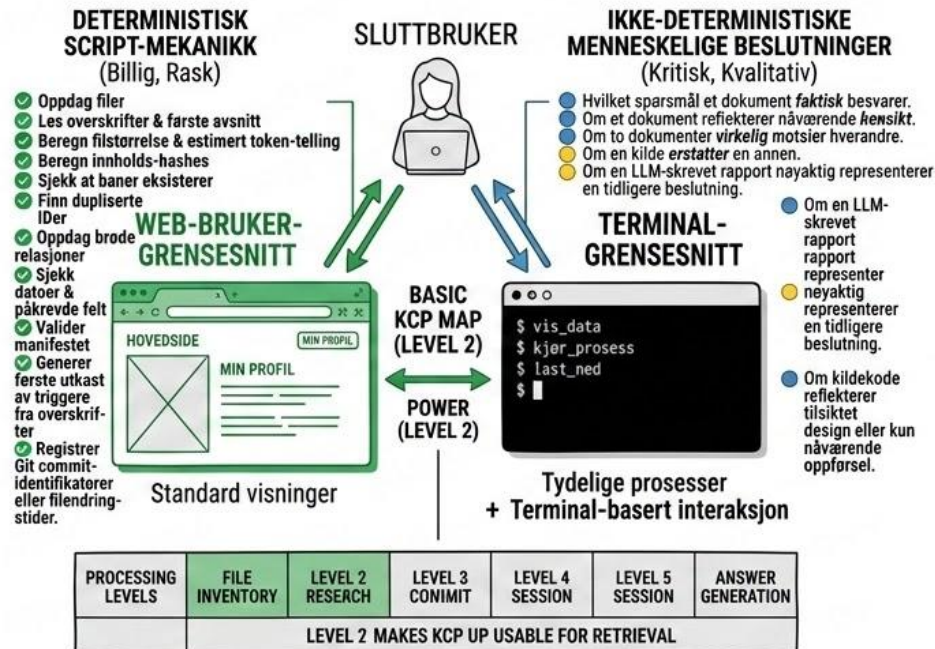
- **KCP:** INDEKSERER OG MAPPER BESKRIVELSER AV INNHOLD, IKKE INNHOLDET SELV.
- **ORIGINALE DOKUMENTER:** FORBLIR UENDRET.
- **MINIMUMSNIVÅ:** EN KNOWLEDGE UNIT PER LOGISK ENHET (Skannet fil, mappe, skill, policy, service, etc.).



Konseptuall-knowledge unit log is å requires:

1. Findang files som kunnskaderer
2. Gruppen grupope uniten
3. Written fragon gliers.
4. Addågen search terms or prosessen.
5. Validaten paths or prosessen.

### OPERASJONELL PROSESSPLITT: SCRIPT VS. MENNESKE



Tjenester – KCP-type informasjon for discovery